

Image Pyramids

Modern Computer Vision | Spring 2026

Oren Chikli | Tutorial 4

This presentation largely relies on BBM 413 Fundamentals of Image Processing, Erkut Erdem, Hacettepe University.

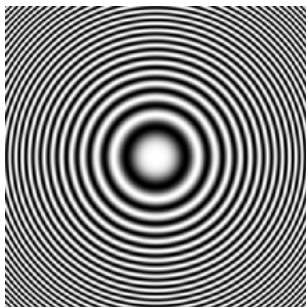
What we already have (from the lecture) — *sampling*

Sampling & Nyquist. To shrink an image safely, low-pass *first*, then sample. Otherwise: aliasing.

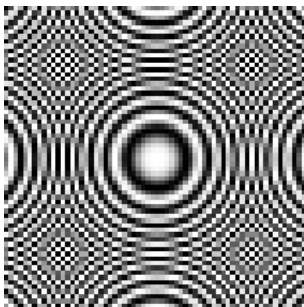
What we already have (from the lecture) — *sampling*

Sampling & Nyquist. To shrink an image safely, low-pass *first*, then sample. Otherwise: aliasing.

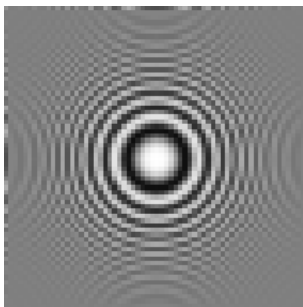
Original
(continuous-like)



Naive subsample $\downarrow 8$
ALIASING (moiré)



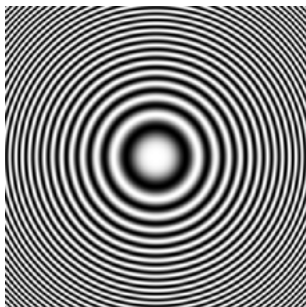
LPF then subsample $\downarrow 8$
clean



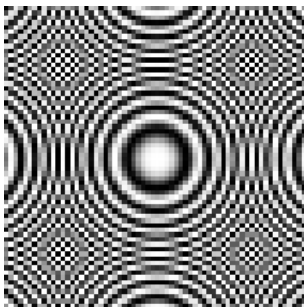
What we already have (from the lecture) — *sampling*

Sampling & Nyquist. To shrink an image safely, low-pass *first*, then sample. Otherwise: aliasing.

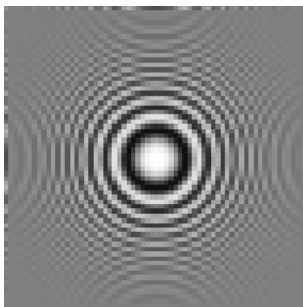
Original
(continuous-like)



Naive subsample $\downarrow 8$
ALIASING (moiré)



LPF then subsample $\downarrow 8$
clean



Naive subsampling folds high spatial frequencies into the low band — the moiré in the middle panel. The Gaussian low-pass on the right removes those frequencies *before* subsampling;

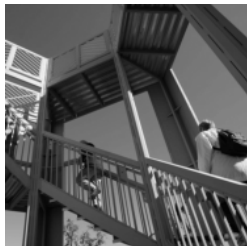
What we already have (from the lecture) — *scale-spaces*

Scale-spaces. Blurring at increasing σ gives a family of images at different scales — but the pixel grid never shrinks, so each blurry copy still costs as much to store and process as the original.

What we already have (from the lecture) — *scale-spaces*

Scale-spaces. Blurring at increasing σ gives a family of images at different scales — but the pixel grid never shrinks, so each blurry copy still costs as much to store and process as the original.

original
512×512



$\sigma = 2$
512×512



$\sigma = 4$
512×512



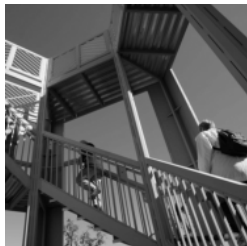
$\sigma = 8$
512×512



What we already have (from the lecture) — *scale-spaces*

Scale-spaces. Blurring at increasing σ gives a family of images at different scales — but the pixel grid never shrinks, so each blurry copy still costs as much to store and process as the original.

original
512×512



$\sigma = 2$
512×512



$\sigma = 4$
512×512



$\sigma = 8$
512×512



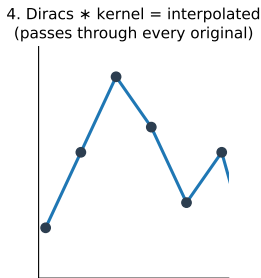
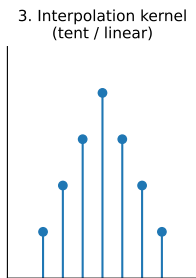
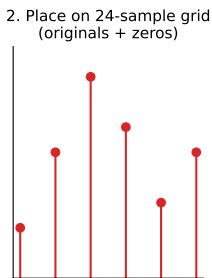
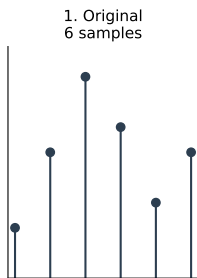
The $\sigma=8$ image is band-limited to $\sim 1/16$ cycles/pixel — by Nyquist it could live on a 32×32 grid with no information loss. We're storing $\sim 256\times$ too many samples.

What we already have (from the lecture) — *resizing*

Resizing = continuous conv with pixel Diracs. To enlarge, place samples as δ 's on the finer grid and convolve with a kernel. That's interpolation.

What we already have (from the lecture) — *resizing*

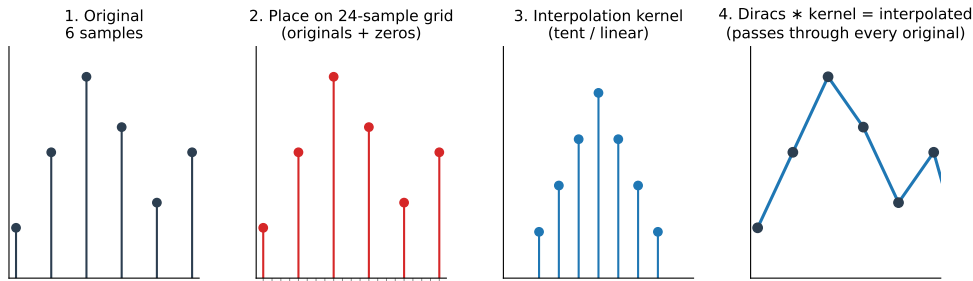
Resizing = continuous conv with pixel Diracs. To enlarge, place samples as δ 's on the finer grid and convolve with a kernel. That's interpolation.



Shoher, Feinstein, Haim, Irani 2020.

What we already have (from the lecture) — *resizing*

Resizing = continuous conv with pixel Diracs. To enlarge, place samples as δ 's on the finer grid and convolve with a kernel. That's interpolation.



Different kernel choices give different interpolators (nearest-neighbour, bilinear, bicubic, sinc, ...). We'll meet a particular 5-tap kernel later that does *both* shrinking and enlarging.

Shocher, Feinstein, Haim, Irani 2020.

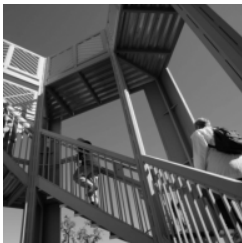
What is an image pyramid?

Image information lives at **many different spatial scales** — edges, textures, objects, whole scenes. An **image pyramid** is a multi-resolution representation: a stack of copies of the image at progressively coarser scales.

What is an image pyramid?

Image information lives at **many different spatial scales** — edges, textures, objects, whole scenes. An **image pyramid** is a multi-resolution representation: a stack of copies of the image at progressively coarser scales.

level 0
512×512



level 1
256×256



level 2
128×128



level 3
64×64



What is an image pyramid?

Image information lives at **many different spatial scales** — edges, textures, objects, whole scenes. An **image pyramid** is a multi-resolution representation: a stack of copies of the image at progressively coarser scales.



Useful for analysing and manipulating images *across* a range of spatial scales — and (as we'll see) for storing them compactly.

Two image pyramids today

- Gaussian pyramid
- Laplacian pyramid

Slide credit: B. Freeman and A. Torralba

Two image pyramids today

- Gaussian pyramid
- Laplacian pyramid

We start here.

Slide credit: B. Freeman and A. Torralba

Two image pyramids today

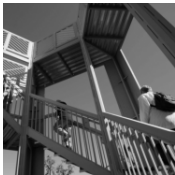
- Gaussian pyramid
- **Laplacian pyramid**

Then the difference between adjacent Gaussian levels.

Slide credit: B. Freeman and A. Torralba

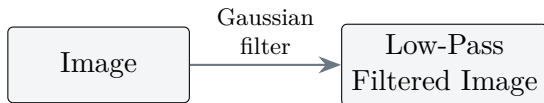
Review of sampling

Image

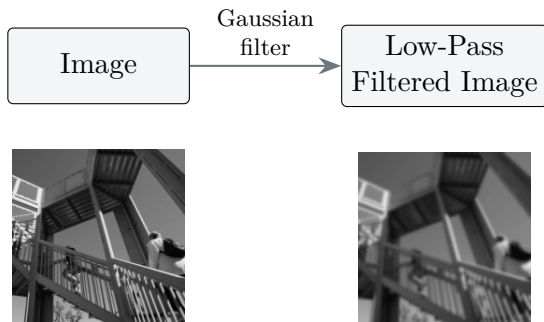


Slide: Hoiem

Review of sampling

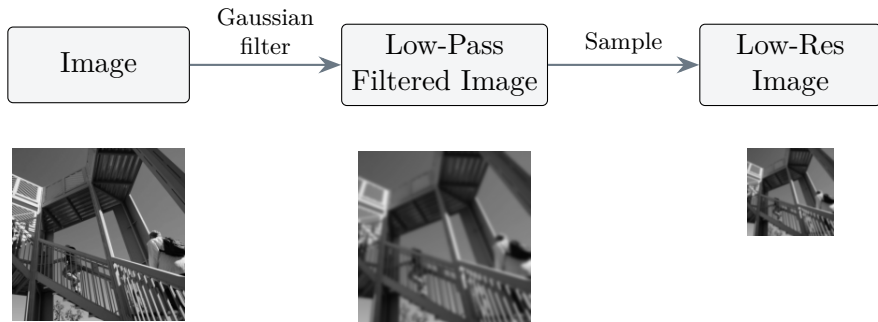


Review of sampling



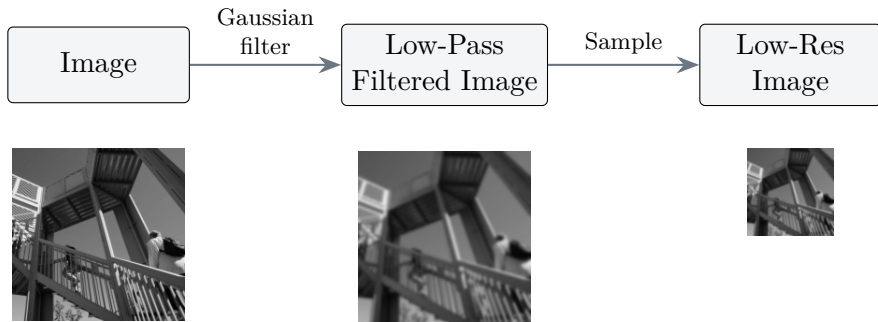
- Filter **before** downsampling — otherwise aliasing (high frequencies fold back into the low band).

Review of sampling



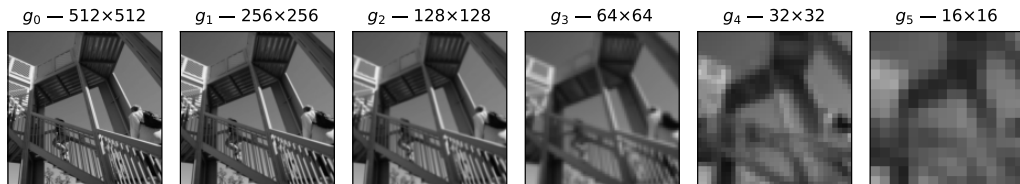
- Filter **before** downsampling — otherwise aliasing (high frequencies fold back into the low band).

Review of sampling



- Filter **before** downsampling — otherwise aliasing (high frequencies fold back into the low band).
- The pyramid is just this *repeated*.

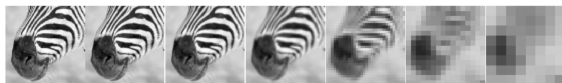
The Gaussian pyramid — example



- g_0 is the original; each level halves both dimensions.
- As we climb the pyramid, fine detail disappears *smoothly* — no aliasing, because each step low-pass filters *first*.

Generated with `pyrtools.GaussianPyramid` on `scipy.datasets.ascent`.

The Gaussian pyramid — another view



512

256

128

64

32

16

8



Fine detail disappears first — the bristles, then the stripes, then everything but the global blob.

Slide credit: B. Freeman and A. Torralba

Why Gaussians? Because $\text{Gauss} * \text{Gauss} = \text{Gauss}$

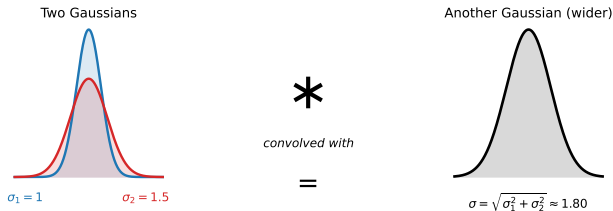
Convolving two Gaussians gives another Gaussian — with a wider σ :

Slide adapted from B. Freeman and A. Torralba

Why Gaussians? Because Gauss * Gauss = Gauss

Convolving two Gaussians gives another Gaussian — with a wider σ :

$$G_{\sigma_1} * G_{\sigma_2} = G_{\sigma} \quad \text{with} \quad \sigma^2 = \sigma_1^2 + \sigma_2^2$$



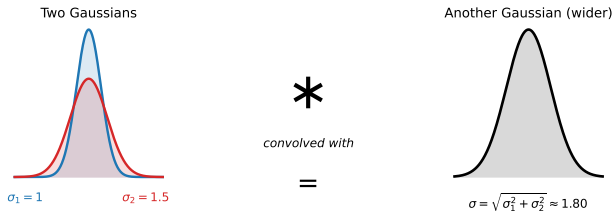
Reminder: you proved this in HW Assignment 1.

Slide adapted from B. Freeman and A. Torralba

Why Gaussians? Because Gauss * Gauss = Gauss

Convolving two Gaussians gives another Gaussian — with a wider σ :

$$G_{\sigma_1} * G_{\sigma_2} = G_{\sigma} \quad \text{with} \quad \sigma^2 = \sigma_1^2 + \sigma_2^2$$

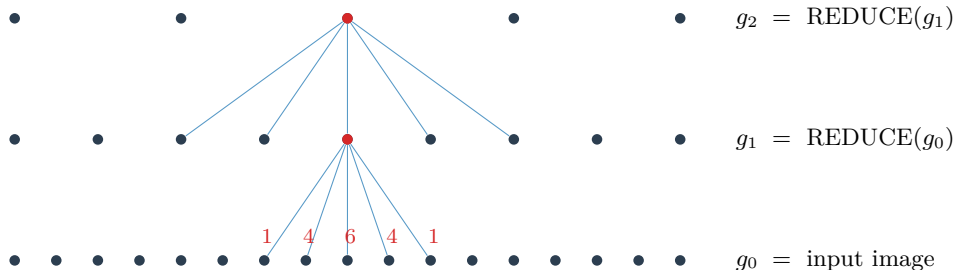


Reminder: you proved this in HW Assignment 1.

Why this matters for pyramids: applying the same Gaussian kernel at every pyramid level still gives a clean Gaussian smoothing of progressively wider effective σ . One kernel, all the way down, with the right blur at every scale.

Slide adapted from B. Freeman and A. Torralba

The computational advantage of pyramids



1D kernel $k = [1, 4, 6, 4, 1] / 16$, reused at every level.

Each level: a local 5-tap weighted average of the level below. Sample count halves each step, so total work is $N + N/4 + N/16 + \dots = O(N)$.

After Burt & Adelson, IEEE Trans. Comm. 1983, Fig. 1.

From here on: 1D notation, 2D pictures

The kernel we will use is $k = [1, 4, 6, 4, 1]/16$, but images are 2D — so why this 1D kernel? Because the matching 2D kernel K is **separable**: it factors as $K = k k^\top$.

$$K = \frac{1}{256} \begin{pmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{pmatrix}$$

So 2D convolution with K = 1D conv with k along rows, then along columns. We work in 1D from here; everything extends to 2D by doing it twice.

Convolution and subsampling as a matrix multiply (1D)

$$x_2 = G_1 x_1 \quad (\text{input length } 19 \rightarrow \text{output length } 8)$$

$$\frac{1}{16} \begin{pmatrix} 1 & 4 & 6 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 4 & 6 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 4 & 6 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 4 & 6 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 4 & 6 & 4 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 4 & 6 & 4 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 4 & 6 & 4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 4 & 6 & 4 & 1 \end{pmatrix}$$

Row i is the recipe for $x_2[i]$. E.g. row 2:

$$x_2[2] = \frac{1}{16} (1 \cdot x_1[4] + 4 \cdot x_1[5] + 6 \cdot x_1[6] + 4 \cdot x_1[7] + 1 \cdot x_1[8]).$$

Rows shift right by **two** columns per step — that's the subsampling baked into the matrix.

Slide credit: B. Freeman and A. Torralba

Next pyramid level

$$x_3 = G_2 x_2 \quad (\text{same kernel, smaller matrix})$$

$$\frac{1}{16} \begin{pmatrix} 1 & 4 & 6 & 4 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 4 & 6 & 4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 4 & 6 & 4 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 4 \end{pmatrix}$$

- Same $[1\ 4\ 6\ 4\ 1]/16$ kernel, same stride-2 — input is just smaller.
- The pyramid is built by **applying G over and over**, on progressively smaller signals. One kernel, all the way down.

Slide credit: B. Freeman and A. Torralba

The combined effect of two pyramid levels

$$x_3 = G_2 G_1 x_1 \quad (\text{input length } 19 \rightarrow \text{output length } 4)$$

$$G_2 G_1 = \frac{1}{256} \begin{pmatrix} 1 & 4 & 10 & 20 & 31 & 40 & 44 & 40 & 31 & 20 & 10 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 4 & 10 & 20 & 31 & 40 & 44 & 40 & 31 & 20 & 10 & 4 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 4 & 10 & 20 & 31 & 40 & 44 & 40 & 30 & 16 & 4 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 4 & 10 & 20 & 25 & 16 & 4 \end{pmatrix}$$

Same row-as-recipe story, but now each recipe is wider.

- Each row contains 13 non-zero weights — the convolution of $[1 \ 4 \ 6 \ 4 \ 1]$ with a stride-2 stretched copy of itself. Same Gaussian-like shape, just **wider**.
- Rows shift by **four** columns per step ($= 2 \times 2$: one stride-2 from G_1 , one more from G_2).
- Going up the pyramid widens the effective filter — exactly what we want for analysis at coarser scales.

Slide credit: B. Freeman and A. Torralba

Equivalent weighting functions

- Row of $G_\ell \cdots G_1 =$ weights $h_\ell(x)$ on the input.

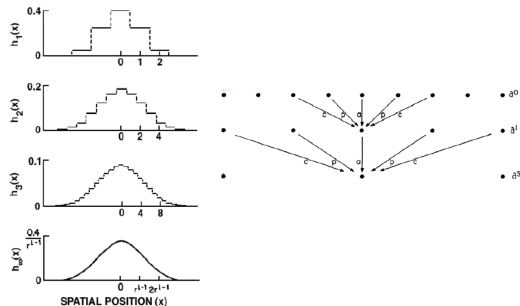


Fig. 2. The equivalent weighting functions $h_\ell(x)$ for nodes in levels 1, 2, 3, and infinity of the Gaussian pyramid. Note that axis scales have been adjusted by factors of 2 to aid comparison. Here the parameter a of the generating kernel is 0.4, and the resulting equivalent weighting functions closely resemble the Gaussian probability density functions.

Slide credit: B. Freeman and A. Torralba

Equivalent weighting functions

- Row of $G_\ell \cdots G_1 =$ weights $h_\ell(x)$ on the input.
- h_1 : 5 bars. h_2 : 13 bars. h_ℓ : wider, smoother.

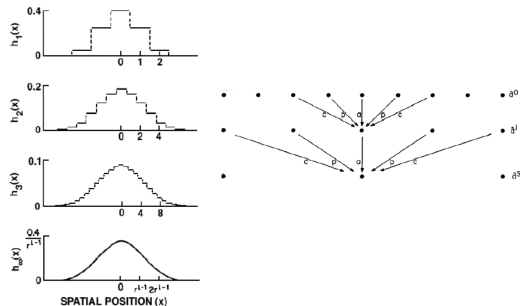


Fig. 2. The equivalent weighting functions $h_\ell(x)$ for nodes in levels 1, 2, 3, and infinity of the Gaussian pyramid. Note that axis scales have been adjusted by factors of 2 to aid comparison. Here the parameter a of the generating kernel is 0.4, and the resulting equivalent weighting functions closely resemble the Gaussian probability density functions.

Slide credit: B. Freeman and A. Torralba

Equivalent weighting functions

- Row of $G_\ell \cdots G_1 =$ weights $h_\ell(x)$ on the input.
- h_1 : 5 bars. h_2 : 13 bars. h_ℓ : wider, smoother.
- Limit $\ell \rightarrow \infty$: Gaussian (CLT).

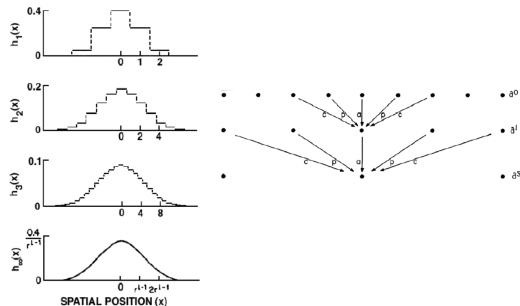


Fig. 2. The equivalent weighting functions $h_\ell(x)$ for nodes in levels 1, 2, 3, and infinity of the Gaussian pyramid. Note that axis scales have been adjusted by factors of 2 to aid comparison. Here the parameter a of the generating kernel is 0.4, and the resulting equivalent weighting functions closely resemble the Gaussian probability density functions.

Slide credit: B. Freeman and A. Torralba

Equivalent weighting functions

- **Row** of $G_\ell \cdots G_1 =$ weights $h_\ell(x)$ on the input.
- h_1 : 5 bars. h_2 : 13 bars. h_ℓ : wider, smoother.
- Limit $\ell \rightarrow \infty$: Gaussian (CLT).
- **Why this matters:**
 - Pyramid = sampled Gaussian scale-space.
 - Each level = faithful image at scale σ_ℓ .

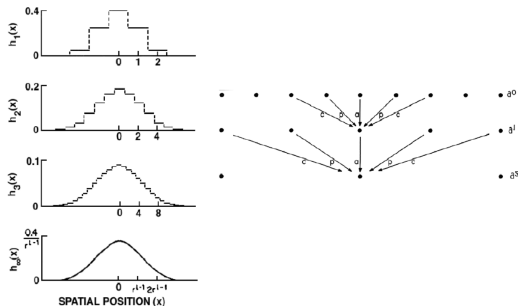


Fig. 2. The equivalent weighting functions $h_\ell(x)$ for nodes in levels 1, 2, 3, and infinity of the Gaussian pyramid. Note that axis scales have been adjusted by factors of 2 to aid comparison. Here the parameter a of the generating kernel is 0.4, and the resulting equivalent weighting functions closely resemble the Gaussian probability density functions.

Slide credit: B. Freeman and A. Torralba

All REDUCE operators in one matrix M

So far we used three separate matrices for the three pyramid levels:

$$\underbrace{I}_{N \times N} : g_0 = I x_1, \quad \underbrace{G_1}_{N/2 \times N} : g_1 = G_1 x_1, \quad \underbrace{G_2 G_1}_{N/4 \times N} : g_2 = G_2 G_1 x_1.$$

Stack them on top of each other:

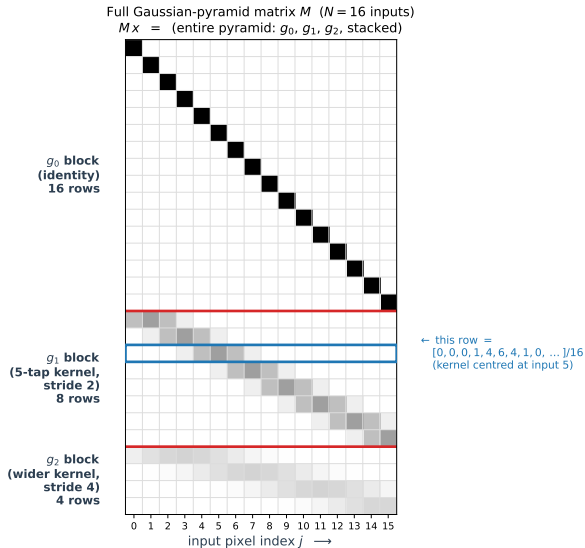
$$M = \begin{bmatrix} I \\ G_1 \\ G_2 G_1 \end{bmatrix}, \quad M x_1 = \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix}.$$

One matrix, $(N + \frac{N}{2} + \frac{N}{4}) \times N = \frac{7N}{4} \times N$ in size = whole pyramid in one multiply.

Why are we showing you this?

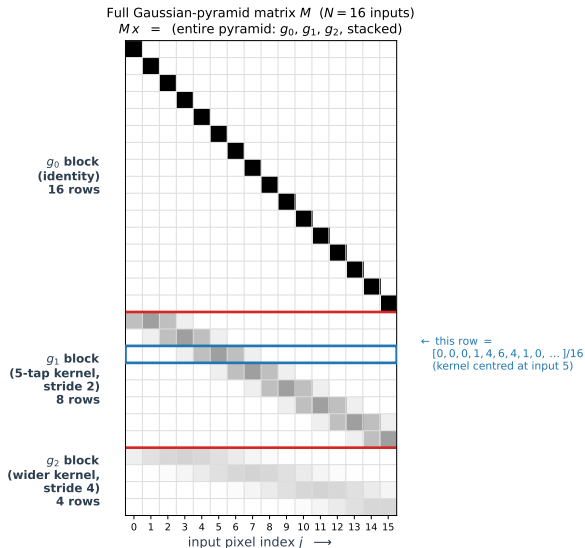
- It unifies the three previous matrix slides into one picture.
- Pyramid construction is a single *linear* operator — the right object to analyse (Burt & Adelson's original framing).

Visualise M as an image



Visualise M as an image

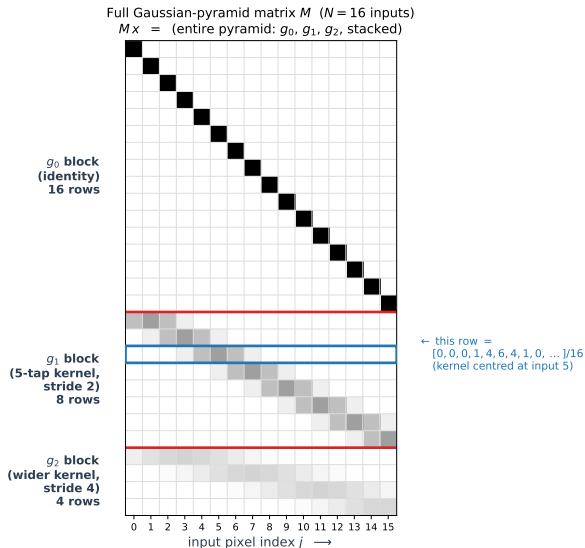
Each **row** = the *receptive field* of one output pixel — which inputs contribute, with what weights.



Visualise M as an image

Each **row** = the *receptive field* of one output pixel — which inputs contribute, with what weights.

Going up the pyramid, the receptive field **widens** (averaging more inputs $\Rightarrow$ smoothing) and adjacent receptive fields are **spaced further apart** (lower resolution).



Gaussian pyramids are used for...

- Up- or down-sampling images cleanly.

Slide credit: B. Freeman and A. Torralba

Gaussian pyramids are used for...

- Up- or down-sampling images cleanly.
- Multi-resolution image analysis:
 - Look for an object over various spatial scales (template matching).

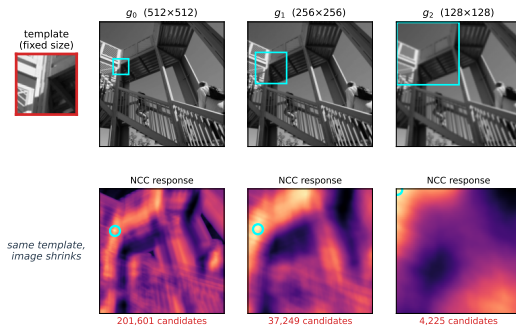
Slide credit: B. Freeman and A. Torralba

Gaussian pyramids are used for...

- Up- or down-sampling images cleanly.
- Multi-resolution image analysis:
 - Look for an object over various spatial scales (template matching).
 - Coarse-to-fine processing: form a blur or motion estimate on a very low-resolution image, upsample, repeat.

Slide credit: B. Freeman and A. Torralba

Template matching with image pyramids



Input: image + template.

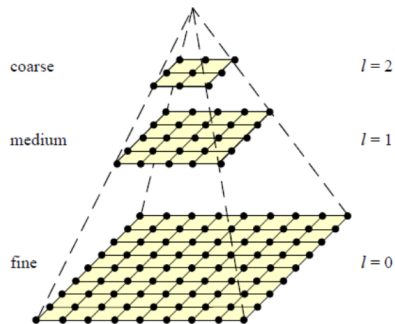
- 1 Match template at the current scale.
- 2 Downsample the image one pyramid step; repeat until small.
- 3 Threshold responses; optional non-maxima suppression.

Same fixed-size template at every level — at coarser levels the 64-px template covers a $2\times / 4\times$ larger physical region, so we automatically search for the object at multiple scales. Fewer candidates per level: 201,601 $\rightarrow$ 37,249 $\rightarrow$ 4,225.

Coarse-to-fine image registration

Algorithm:

- 1 Compute Gaussian pyramids of both images.
- 2 Align at the coarsest pyramid level.
- 3 Successively align with finer pyramids — search a smaller range each level.



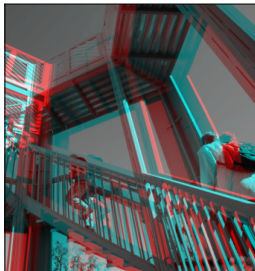
Slide credit: B. Freeman and A. Torralba

Coarse-to-fine registration — the idea

Setup. Image A and image $B = A$ shifted by an unknown Δ pixels. Find Δ .

- Brute-force at full resolution: search every offset in ± 32 px $\Rightarrow 65 \times 65 = 4225$ candidates. Expensive.
- **Key observation.** The same physical shift is *smaller in pixels at coarser levels* — each pyramid step halves it.

g_0 (480×512)



shift = 32 pixels

± 32 -px search = 4225 candidates

g_3 (60×64)

blur & subsample
 $\times 3 \rightarrow$

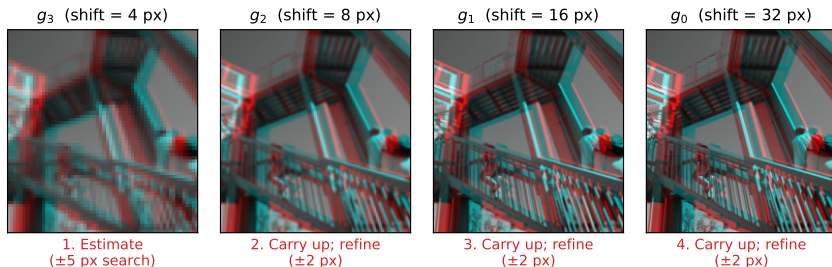


shift = 4 pixels

± 5 -px search = 121 candidates

Coarse-to-fine registration — the algorithm

- 1 Build Gaussian pyramids of A and B .
- 2 At g_3 : brute-force ± 5 search $\Rightarrow \Delta_3 \approx 4$. (121 candidates)
- 3 Carry up ($\Delta_{\ell-1} = 2\Delta_\ell$); refine with ± 2 at each finer level. (25 each)



Total = $121 + 3 \times 25 = 196$ candidates vs 4225 brute-force — $\sim 22\times$ faster.

Two image pyramids today

- Gaussian pyramid

(done)

- **Laplacian pyramid**

The Gaussian pyramid mixes frequency bands at every level. The Laplacian pyramid separates them.

Slide credit: B. Freeman and A. Torralba

The Laplacian pyramid

Idea. At level ℓ , store the *difference* between the current Gaussian level and a *prediction* of it built from the next-coarser level:

$$L_\ell = g_\ell - \text{EXPAND}(g_{\ell+1}).$$

- Each L_ℓ is a **band-pass** image — it captures one *octave* of spatial frequencies.
- Together, $L_0, L_1, \dots, L_N, g_N$ **exactly** reconstruct g_0 . (We'll see why.)

Slide adapted from B. Freeman and A. Torralba

EXPAND — what is it, mechanically?

Goal: take a small image $g_{\ell+1}$ and produce a larger image at the same resolution as g_ℓ , so we can *subtract*.

Two steps:

- 1 **Insert zeros** between every adjacent pair of samples (and on every other row/column in 2D). The grid is now $2\times$ as wide.
- 2 **Apply the same $[1\ 4\ 6\ 4\ 1]/16$ kernel** ($\times$ gain factor 2 in 1D, 4 in 2D).

EXPAND — 1D mini-example

Start with a tiny 1D signal $g_2 = [29.8, 42.7, 29.8]$.

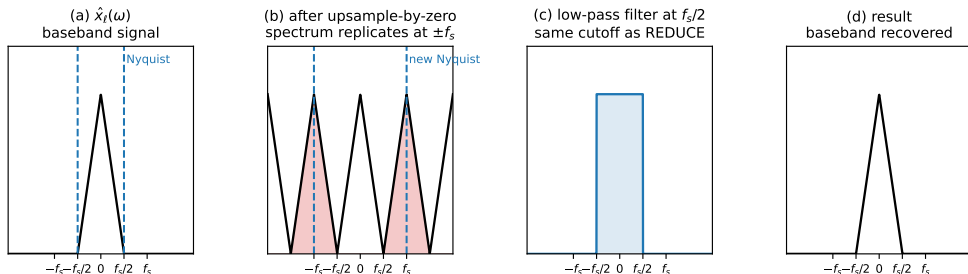
- **Insert zeros:** $[29.8, 0, 42.7, 0, 29.8]$.
- **Filter** with $[1, 4, 6, 4, 1]/16 \times 2$: $[32.4, 36.3, 40.1, 36.3, 32.4]$.
- The zeros got filled by interpolation; the kernel averaged across them.

The gain factor 2 compensates for the energy lost to the inserted zeros — without it the output would be half as bright as the input.

Why the *same* kernel? — frequency view

Recall: **sampling at rate f_s in time $\Leftrightarrow$ spectrum replication at multiples of f_s in frequency.**

Inserting zeros doubles the sample rate, so the spectrum *replicates* — we need an LPF at the *old* Nyquist to kill the copies:



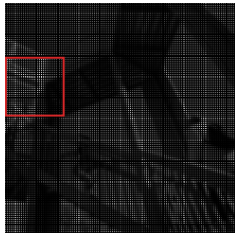
Same cutoff at $f_s/2$ in both directions — so REDUCE and EXPAND **reuse the same kernel**.

What “imaging” looks like — image view

input (128×128)



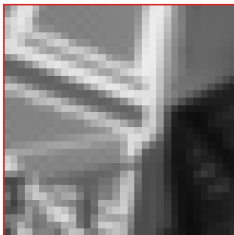
zero-inserted, NO filter
(3/4 of pixels are 0 — stippled)



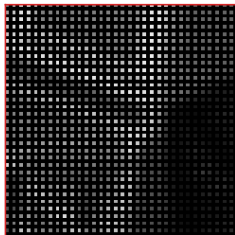
zero-inserted + EXPAND filter
(clean upsampling, gain ×4)



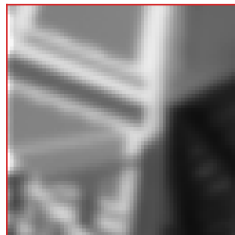
zoom (red box)



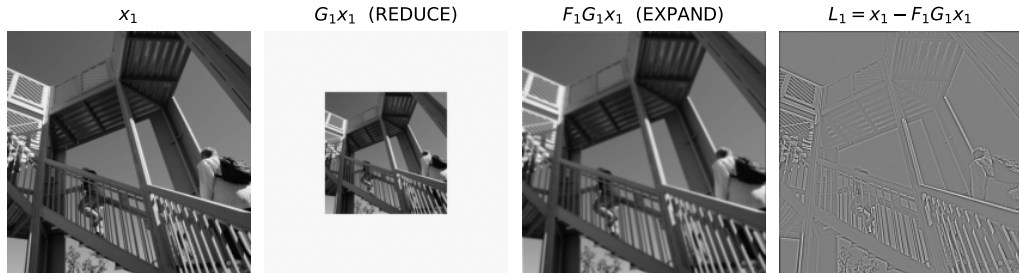
zoom (red box)



zoom (red box)



Building L_1 — see what each operator does



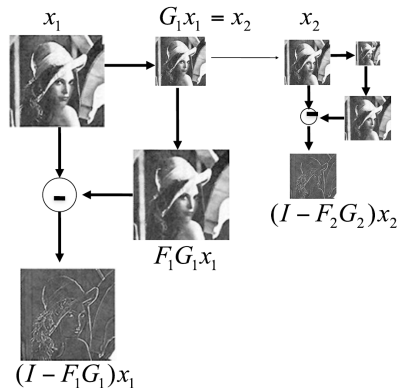
- G_1x_1 **shrinks** (smaller and blurry) — the next Gaussian level.
- $F_1G_1x_1$ **stretches back** (same size, still blurry) — a prediction of x_1 from g_2 .
- L_1 is the **residual**: what the prediction missed — the high-frequency detail. Histogram is sharply peaked at zero (most pixels are grey).

Repeat at every level

- Same recipe on $x_2 = G_1 x_1$, then $x_3 = G_2 x_2, \dots$
- Stack the residuals $L_1, L_2, L_3, \dots$ plus the tiny Gaussian top g_N .
- One **octave** of frequencies per level.

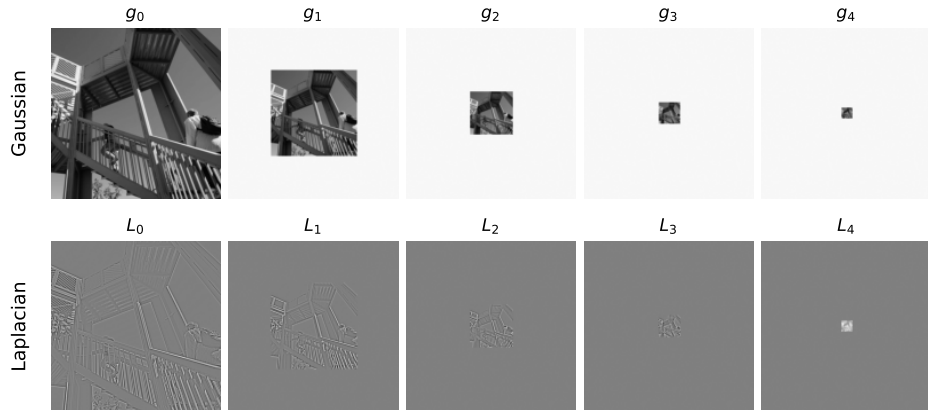
octave = frequency band with upper bound $2\times$ the lower bound.

The Laplacian Pyramid



Slide credit: B. Freeman and A. Torralba

Gaussian (top) vs. Laplacian (bottom) — same image



- g_ℓ : blurrier copies, but *redundant* (each contains the low frequencies of the next).
- L_ℓ : the band *thrown away* between consecutive Gaussian levels — sparse, near-zero-mean (grey). L_0 has sharp edges, deeper levels have coarser structure (one octave per level).

Laplacian reconstruction — recover x_1 from L_1, L_2, L_3, x_4

$G_{\#}$ is the blur-and-downsample operator at pyramid level $\#$.

$F_{\#}$ is the blur-and-upsample operator at pyramid level $\#$.

Band-pass coefficients (output):

$$L_1 = (I - F_1 G_1) x_1$$

$$L_2 = (I - F_2 G_2) x_2$$

$$L_3 = (I - F_3 G_3) x_3$$

Reconstruct x_1 (bottom-up):

$$x_3 = L_3 + F_3 x_4$$

$$x_2 = L_2 + F_2 x_3$$

$$x_1 = L_1 + F_1 x_2$$

Gaussian recurrence (intermediate):

$$x_2 = G_1 x_1$$

$$x_3 = G_2 x_2$$

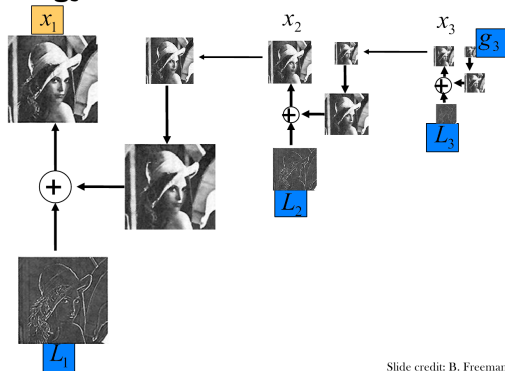
$$x_4 = G_3 x_3$$

Exact reconstruction, level by level.

Slide credit: B. Freeman and A. Torralba

Laplacian reconstruction — visual

**Laplacian pyramid reconstruction
algorithm: recover x_1 from L_1, L_2, L_3
and g_3**



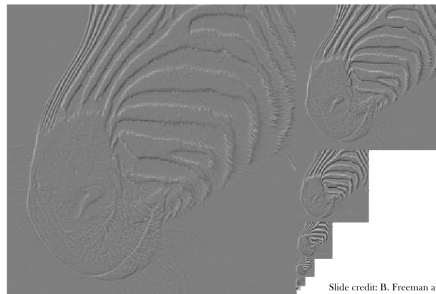
Start from the coarsest Gaussian g_3 , add L_3 to recover x_3 , upsample, add $L_2, \dots$ — climb back to x_1 .

Slide credit: B. Freeman and A. Torralba

Laplacian pyramid — visual example



512 256 128 64 32 16 8



Slide credit: B. Freeman and A. Torralba

Each level is mostly grey (zero), with edge / texture *at that scale* popping out — exactly the band-pass behaviour the algebra promised.

Slide credit: B. Freeman and A. Torralba

The Laplacian Pyramid as a Compact Image Code

Peter J. Burt, MEMBER, IEEE, and Edward H. Adelson

Concrete numbers on “Lady” (257×257):

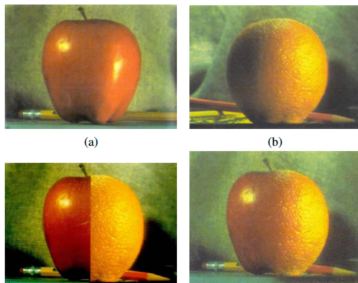
	Raster (8-bit)	Laplacian pyramid
Entropy / bit-rate	7.57 bpp	1.58 bpp
Reconstruction error	—	RMSE $\approx 2.4 / 255$
Sample-count overhead	1.00	$4/3$

The $4/3$ overhead is paid in samples but more than offset by the bits-per-sample drop. Wavelets later removed the overhead entirely.

Burt & Adelson, IEEE Trans. Comm. COM-31(4), 1983.

Image blending

Image blending



Slide credit: B. Freeman and A. Torralba

- Bottom-left: naïve cut-and-paste — visible seam.
- Bottom-right: *multi-band* blending — seamless.

Image blending — by frequency band

Szeliski, Computer Vision, 2010

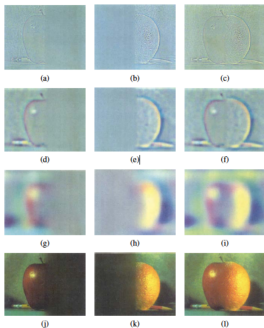


Figure 3.42 Laplacian pyramid blending details (Burt and Adelson 1983b) © 1983 ACM. The first three rows show the high, medium, and low frequency parts of the Laplacian pyramid (taken from levels 0, 2, and 4). The left and middle columns show the original apple and orange images weighted by the smooth interpolation functions, while the right column shows the averaged contributions.

Slide credit:
B. Freeman &
A. Torralba

- Top rows: *high-frequency* pyramid levels — sharp edges, narrow transition needed.
- Bottom rows: *low-frequency* levels — smooth transition, wide blur.
- Each band gets its *own* blending mask width: sharp at high freq, soft at low freq.
- Right column: per-level result.

Szeliski, Computer Vision, 2010 | B. Freeman & A. Torralba

Image blending — algorithm

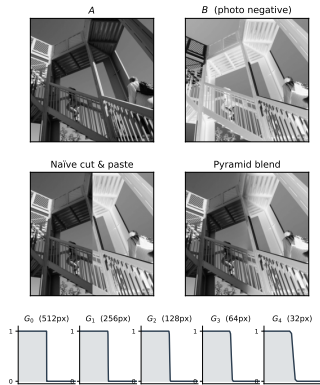
Given two images A , B and a binary mask:

- 1 Build the Laplacian pyramid of each image:
 L_A , L_B .
- 2 Build the *Gaussian* pyramid of the mask: G .
- 3 Combine band by band:

$$L_j = G_j L_A^j + (1 - G_j) L_B^j.$$

- 4 Collapse L (reconstruct from the bottom up) to obtain the blended image.

The Gaussian mask is automatically **wider at coarse levels** and sharper at fine levels — exactly what the previous slide showed.



Mask Gaussian pyramid (horizontal profile): sharp step at g_0 , widening ramp at higher levels.

Slide credit: B. Freeman and A. Torralba

Eulerian Video Magnification

Eulerian Video Magnification

<https://www.youtube.com/watch?v=ONZcjsIPjmk>

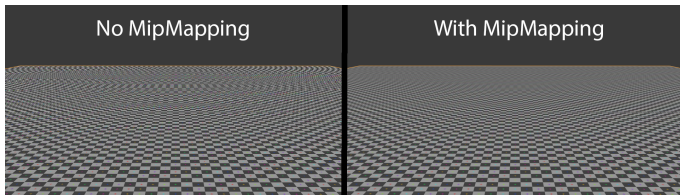


- Same band-pass idea applied *temporally*: amplify changes inside a chosen frequency band at each pyramid level.
- Invisible motions (heartbeat in the face, building sway) become visible.

Wu et al., SIGGRAPH 2012

Mipmaps — the pyramid in your GPU

GPUs store every texture as an image pyramid — a **mipmap**. Distant pixels sample a coarser level; the anti-aliasing argument from this morning, applied per-pixel in hardware.



Same ground plane, same camera. Left: nearest-textel sampling — moiré in the distance. Right: mipmapped — smooth grey.

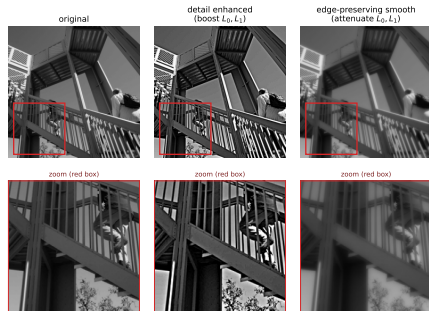
What the GPU stores: prefiltered halvings.

- **Near pixels** sample fine levels; **far pixels** sample coarse levels — the GPU picks the level per-pixel.
- Trilinear filtering blends two adjacent levels — the EXPAND idea, in hardware.
- Memory cost: $1 + \frac{1}{4} + \frac{1}{16} + \dots = \frac{4}{3} \times$ the original texture.

Aliasing comparison: Wikipedia, public domain.

Local Laplacian Filters — edge-aware image editing

Build a Laplacian pyramid, **manipulate the coefficients level-by-level**, reconstruct. Edge-preserving by construction.

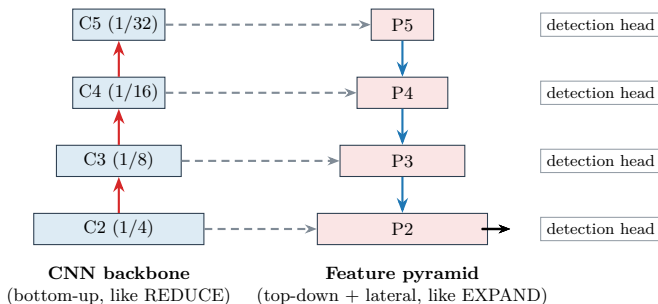


- Boost $L_0, L_1 \Rightarrow$ detail enhancement; attenuate them $\Rightarrow$ edge-aware smoothing.
- Same machinery underlies **HDR tone mapping** and Photoshop's “Clarity / Detail” sliders.

Paris, Hasinoff, Kautz — SIGGRAPH 2011

Feature Pyramid Networks — the pyramid in modern detection

Feature Pyramid Networks (Lin et al. 2017) made the multi-scale move explicit: instead of pyramiding the *input image*, pyramid the *feature maps* a CNN already computes at decreasing resolutions.



Same REDUCE / EXPAND scaffolding as 1983 — only the filters are now *learned*, and detection happens at every level instead of just the top.

Lin et al., "Feature Pyramid Networks for Object Detection", CVPR 2017.

Other modern descendants you've already met (or will)

Even where pyramids aren't named, the pattern is everywhere in modern CV:

- **SIFT / DoG** (Lowe 1999) — keypoint detection on a Laplacian pyramid. Differences-of-Gaussians *are* a Laplacian pyramid. Direct continuation of the 1983 idea.
- **U-Net** (Ronneberger 2015) — segmentation. The encoder–decoder with skip connections is the Laplacian-pyramid analysis/synthesis story with *learned* filters at every level.
- **Swin Transformer** (Liu 2021) and **ConvNeXt** — hierarchical backbones with halving resolutions per stage. The pyramid prior, baked into the architecture.

Every one of these inherits the same insight: process the image at multiple scales, combine across them.

Cascaded Diffusion — the pyramid in image generation

Generating a 1024×1024 image directly is expensive and hard to train. **Cascaded Diffusion** (Ho et al. 2021; Imagen, 2022) chains models across resolutions instead:



- **Coarse-to-fine generation**: the base model decides *what* the image is at 64×64 ; later stages add *detail*. Direct philosophical descendant of coarse-to-fine registration.
- Each super-res stage learns the *Laplacian-like residual* — what's missing between adjacent resolutions.
- Used in **Imagen**, **eDiff-I**, **Stable Diffusion XL refiner**, and recent video models.

Ho et al., "Cascaded Diffusion Models for High-Fidelity Image Generation", JMLR 2022.

Take-aways

- **REDUCE & EXPAND share one kernel.** The 5-tap $[1, 4, 6, 4, 1]/16$ low-pass appears in both directions; what differs is whether it lives in matrix *rows* (REDUCE) or matrix *columns* (EXPAND).
- **Laplacian = band-pass = exact reconstruction.** Each L_ℓ stores one octave of frequencies; summing all L_ℓ plus the coarsest Gaussian rebuilds the original *exactly* — at a $4/3$ sample-count overhead.
- **Pyramids are everywhere in modern CV.** SIFT, U-Net, FPN, mipmaps, Local Laplacian filters, cascaded diffusion — all riff on Burt & Adelson's 1983 idea: process the image at multiple scales, combine across them.

Questions?

See you next week.